

Tarea 3.0 Trabajo Colaborativo en Parejas

Investigación

Control de versiones

- **Definición:** Es un sistema que registra los cambios realizados en uno o varios archivos a lo largo del tiempo, permitiendo recuperar versiones específicas posteriormente. Existen sistemas centralizados (un solo repositorio central) y distribuidos (cada usuario posee una copia completa del historial).
- **Utilidad:** Permite llevar un historial ordenado de cambios, recuperar versiones anteriores ante errores, comparar modificaciones entre fechas o autores, y que varias personas trabajen sobre el mismo proyecto sin sobrescribir el trabajo de los demás.
- **Ejemplo práctico:** Un equipo de desarrollo de software utiliza un sistema de control de versiones para que cada integrante pueda modificar el código en paralelo, sin perder el historial de cambios ni el trabajo de sus compañeros.
- **Fuente:** [Git - About Version Control](#)

Git

- **Definición:** Es un sistema de control de versiones distribuido, gratuito y de código abierto, diseñado para gestionar proyectos de cualquier tamaño con rapidez y eficiencia. Fue creado por Linus Torvalds en 2005.
- **Utilidad:** Permite a los desarrolladores guardar el historial completo de un proyecto en su propio equipo, crear ramas para trabajar en funcionalidades de forma aislada, y combinar (merge) el trabajo de distintas personas de manera ordenada.
- **Ejemplo práctico:** Un desarrollador instala Git en su computadora y ejecuta comandos como git init, git add y git commit para empezar a versionar un proyecto local antes de subirlo a un repositorio remoto.
- **Fuente:** [Git – About](#)

GitHub

- **Definición:** Es una plataforma web que aloja repositorios Git en la nube y agrega funcionalidades de colaboración como Pull Requests, revisión de código, gestión de issues, wikis y automatización mediante GitHub Actions.
- **Utilidad:** Facilita el trabajo en equipo al centralizar el código en un servidor remoto accesible para todos los integrantes del proyecto, y permite aplicar buenas prácticas de revisión y aprobación de cambios antes de integrarlos.
- **Ejemplo práctico:** Un equipo crea un repositorio en GitHub para alojar el código de su aplicación web, y cada integrante sube sus cambios mediante Pull Requests que son revisados antes de fusionarse a la rama principal.
- **Fuente:** [GitHub Docs – What is GitHub?](#)

Repositorio local

- **Definición:** Es la copia del proyecto y su historial de versiones almacenada directamente en la computadora del desarrollador. Se crea mediante git init o al clonar un repositorio remoto con git clone.
- **Utilidad:** Permite trabajar sin conexión a internet, realizar commits, crear ramas y probar cambios antes de compartirlos con el resto del equipo a través del repositorio remoto.
- **Ejemplo práctico:** Un estudiante clona un repositorio con git clone en su computadora, obteniendo así un repositorio local idéntico al remoto, donde puede crear ramas y hacer commits.
- **Fuente:** [Pro Git Book – Getting a Git Repository](#)

Repositorio remoto

- **Definición:** Es una versión del repositorio alojada en un servidor externo, como GitHub, accesible por internet, que permite que múltiples colaboradores compartan y sincronicen su trabajo.

- **Utilidad:** Sirve como punto central de referencia del proyecto: ahí se publican los cambios mediante git push y desde ahí se descargan las actualizaciones de otros colaboradores mediante git pull.
- **Ejemplo práctico:** Un equipo de dos personas comparte un mismo repositorio remoto en GitHub; ambos integrantes publican sus ramas y Pull Requests en ese repositorio para integrarlos a la rama principal.
- **Fuente:** [Pro Git Book – Working with Remotes](#)

Commit

- **Definición:** Es una instantánea (snapshot) de los cambios realizados en los archivos del proyecto en un momento determinado, acompañada de un mensaje descriptivo que explica qué se modificó.
- **Utilidad:** Permite registrar de forma ordenada el progreso del proyecto, identificar qué cambios se hicieron, cuándo y por quién, y volver a una versión anterior específica si es necesario.
- **Ejemplo práctico:** Después de modificar el archivo index.html, el desarrollador ejecuta git commit -m "Agrega título principal en index.html" para guardar ese cambio en el historial del proyecto.
- **Fuente:** [Pro Git Book – Recording Changes to the Repository](#)

Branch (rama)

- **Definición:** Es una línea de desarrollo independiente dentro de un repositorio que permite trabajar en nuevas funcionalidades o correcciones sin afectar el código de la rama principal (main).
- **Utilidad:** Permite que varios desarrolladores trabajen en distintas funcionalidades de forma simultánea y aislada, evitando conflictos hasta que el trabajo esté listo para integrarse mediante un merge.
- **Ejemplo práctico:** Antes de agregar una nueva funcionalidad, un desarrollador ejecuta git checkout -b rama-estudiante-B para crear una rama independiente donde realizar los cambios sin afectar main.

- **Fuente:** [Pro Git Book – Git Branching](#)

Merge

- **Definición:** Es el proceso mediante el cual Git combina los cambios de dos ramas distintas en una sola, integrando el historial y el contenido de ambas.
- **Utilidad:** Permite incorporar el trabajo realizado en una rama secundaria (por ejemplo, una funcionalidad nueva) a la rama principal del proyecto, una vez que ese trabajo ha sido revisado y aprobado.
- **Ejemplo práctico:** Una vez aprobado un Pull Request, el revisor presiona el botón Merge pull request en GitHub, lo que combina los commits de la rama rama-estudiante-B dentro de la rama main.
- **Fuente:** [Pro Git Book – Basic Branching and Merging](#)

Pull Request

- **Definición:** Es una solicitud formal dentro de GitHub para integrar los cambios de una rama hacia otra (normalmente hacia main), permitiendo que el equipo revise, comente y apruebe el código antes de fusionarlo.
- **Utilidad:** Facilita la revisión colaborativa de código, el control de calidad y la discusión de cambios antes de que estos se integren de forma definitiva al proyecto.
- **Ejemplo práctico:** Después de publicar la rama rama-estudiante-A, el desarrollador crea un Pull Request en GitHub para que su compañero revise y apruebe los cambios antes de fusionarlos a main.
- **Fuente:** [GitHub Docs – About Pull Requests](#)

Fork

- **Definición:** Es una copia completa de un repositorio ajeno que se crea dentro de la propia cuenta de GitHub del usuario, permitiendo modificarlo libremente sin afectar el proyecto original.

- **Utilidad:** Es útil para contribuir a proyectos de código abierto en los que el colaborador no tiene permisos directos de escritura: se hace un fork, se modifica, y luego se propone el cambio al proyecto original mediante un Pull Request.
- **Ejemplo práctico:** Un desarrollador externo hace un fork del repositorio de un proyecto open source, realiza mejoras en su copia y posteriormente envía un Pull Request al repositorio original para que sus cambios sean considerados.
- **Fuente:** [GitHub Docs – Fork a repo](#)

Conflictos de integración

- **Definición:** Ocurren cuando Git no puede combinar automáticamente los cambios de dos ramas porque ambas modificaron la misma parte de un archivo de forma diferente, requiriendo que el desarrollador decida manualmente cómo unificar el contenido.
- **Utilidad:** Comprender cómo identificar y resolver conflictos es esencial en el trabajo en equipo, ya que es habitual que dos personas modifiquen el mismo archivo; resolverlos correctamente evita la pérdida de cambios importantes.
- **Ejemplo práctico:** Dos ramas distintas modifican la misma línea del archivo index.html; al intentar fusionarlas, Git marca el conflicto con las etiquetas <<<<<<, ===== y >>>>>>, y el desarrollador edita el archivo manualmente para resolverlo antes de hacer commit.
- **Fuente:** [GitHub Docs – Resolving a merge conflict](#)

Integración Continua (CI)

- **Definición:** Es una práctica de desarrollo en la que los integrantes de un equipo integran su código en un repositorio compartido de forma frecuente, validando automáticamente cada cambio mediante compilaciones y pruebas automatizadas.
- **Utilidad:** Permite detectar errores de forma temprana, reducir los problemas de integración entre distintas ramas y mantener el código del repositorio en un estado funcional de manera constante.

- **Ejemplo práctico:** Cada vez que se sube un cambio a la rama main, un workflow de GitHub Actions se ejecuta automáticamente para verificar que el código no tenga errores antes de aceptarlo como válido.
- **Fuente:** [GitHub Docs – About continuous integration](#)

Entrega Continua (Continuous Delivery)

- **Definición:** Es la práctica que extiende la integración continua asegurando que el código, después de pasar todas las pruebas automáticas, quede siempre en un estado listo para ser desplegado a producción, aunque el despliegue final requiera una aprobación manual.
- **Utilidad:** Reduce el tiempo y el riesgo de liberar nuevas versiones del software, ya que el proceso de preparación para el despliegue está automatizado y solo falta una decisión humana para publicarlo.
- **Ejemplo práctico:** Un equipo configura su pipeline para que, tras pasar las pruebas automáticas, el código quede empaquetado y listo; un responsable solo debe aprobar manualmente la publicación final a producción.
- **Fuente:** [GitHub Docs – CI/CD concepts](#)

Despliegue Continuo (Continuous Deployment)

- **Definición:** Es una práctica aún más avanzada que la entrega continua, en la que cada cambio que pasa exitosamente todas las pruebas automatizadas se despliega automáticamente a producción, sin necesidad de aprobación manual.
- **Utilidad:** Permite que las nuevas funcionalidades y correcciones lleguen a los usuarios finales de forma inmediata y constante, acelerando el ciclo de entrega de valor del software.
- **Ejemplo práctico:** Una aplicación web configurada con despliegue continuo publica automáticamente en el servidor de producción cada vez que un Pull Request se fusiona exitosamente a la rama main y pasa todas las pruebas.
- **Fuente:** [GitHub Docs – CI/CD concepts](#)

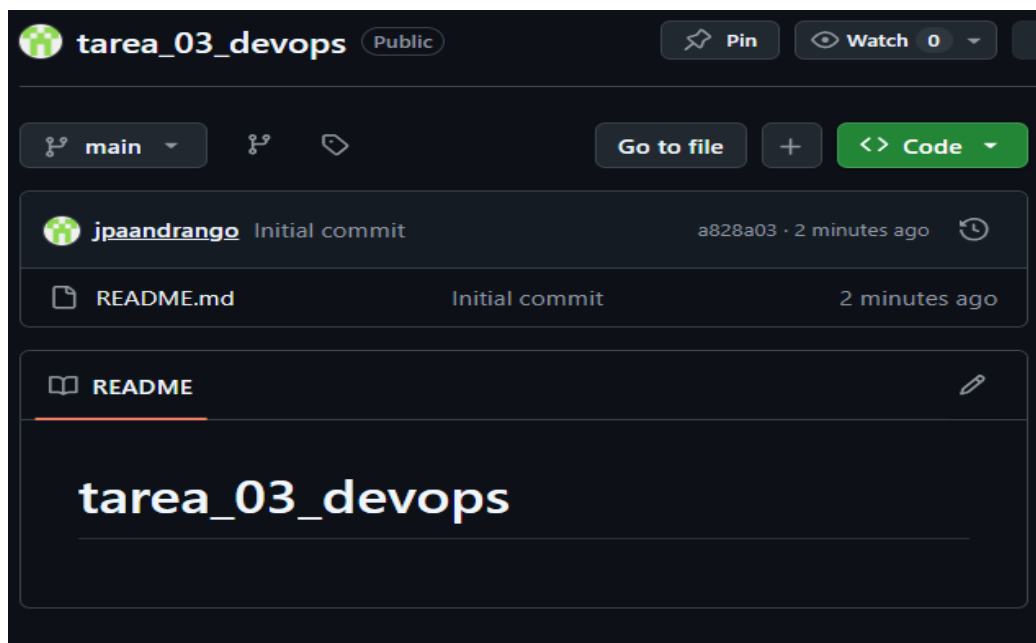
GitHub Actions

- **Definición:** Es la herramienta de automatización integrada en GitHub que permite crear flujos de trabajo (workflows) definidos en archivos YAML para ejecutar tareas automáticas, como pruebas, compilaciones o despliegues, ante eventos como un push o un Pull Request.
- **Utilidad:** Permite automatizar procesos de integración y entrega continua directamente dentro del repositorio de GitHub, sin necesidad de herramientas externas, ejecutándose automáticamente cada vez que ocurre el evento configurado.
- **Ejemplo práctico:** Se crea el archivo `.github/workflows/main.yml` configurado para ejecutarse en cada push a main; al fusionar un Pull Request, GitHub Actions ejecuta automáticamente el workflow y muestra el resultado (éxito o error) en la pestaña Actions del repositorio.
- **Fuente:** [GitHub Docs – Understanding GitHub Actions](#)

Demostración Práctica

Creación del repositorio en GitHub

Se creó el repositorio público “tarea_03_devops” en GitHub (rol Estudiante A), inicializándolo con un archivo README.md mediante un commit inicial, este repositorio sirvió como punto central remoto para todo el flujo de trabajo colaborativo posterior.



Creación de rama, commit y publicación de cambios — Estudiante B

Actuando como Estudiante B, se creó la rama `rama-estudiante-B` con el comando `git checkout -b`, se agregó un título principal en el archivo `index.html`, se realizó el commit con un mensaje descriptivo (“Agrega título principal en `index.html`”) y finalmente se publicó la rama en el repositorio remoto mediante `git push origin rama-estudiante-B`.

```
PS C:\Users\User\Desktop\DEVOPS\tarea_03> cd estudiante-B
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> git checkout -b rama-estudiante-B
Switched to a new branch 'rama-estudiante-B'
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> echo "<h1>Mi practica de Git y GitHub</h1>" > index.html
>>
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> git add .
>>
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> git commit -m "Agrega titulo principal en index.html"
>>
[rama-estudiante-B ea726c4] Agrega titulo principal en index.html
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 index.html
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> git push origin rama-estudiante-B
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 359 bytes | 359.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'rama-estudiante-B' on GitHub by visiting:
remote:   https://github.com/jpaandrango/tarea_03_devops/pull/new/rama-estudiante-B
remote:
To https://github.com/jpaandrango/tarea_03_devops.git
 * [new branch]      rama-estudiante-B -> rama-estudiante-B
```

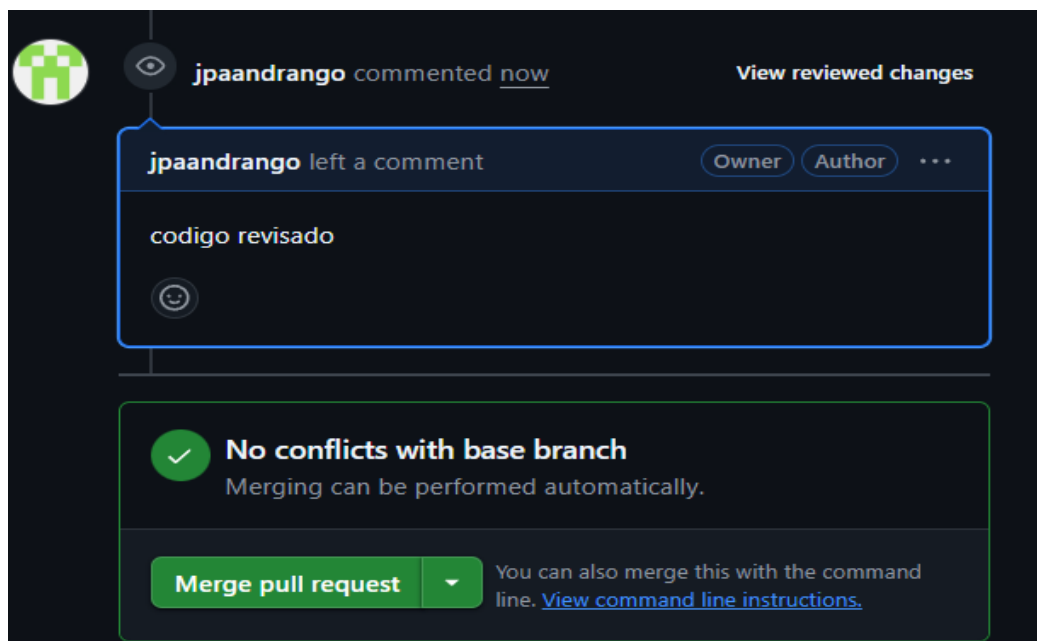
Creación del Pull Request — Estudiante B

Una vez publicada la rama, se generó el Pull Request #1 “Agrega título principal en `index.html`” desde GitHub, solicitando integrar la rama `rama-estudiante-B` hacia `main`. GitHub confirmó que no existían conflictos con la rama base y que la fusión podía realizarse automáticamente.



Revisión de código — Estudiante A

Cambiando al rol de Estudiante A, se revisó el código propuesto en el Pull Request #1 y se dejó un comentario de revisión "codigo revisado" en la pestaña de cambios, confirmando que el contenido era correcto antes de proceder con la aprobación.



Aprobación y Merge del Pull Request #1 — Estudiante A

Tras revisar el código, el Estudiante A aprobó el Pull Request y ejecutó el proceso de Merge, integrando el commit de la rama `rama-estudiante-B` hacia la rama `main`. GitHub confirmó el estado “Merged” y el cierre exitoso del Pull Request.

The screenshot displays a GitHub Pull Request interface. At the top, the title "Agrega titulo principal en index.html #1" is shown with an edit icon. Below the title, a purple "Merged" badge is present, followed by the text "jpaandrango merged 1 commit into `main` from `rama-estudiante-B` now". A navigation bar includes tabs for "Conversation" (1), "Commits" (1), "Checks" (0), and "Files changed" (1). The main content area shows a commit history with a commit titled "Agrega titulo principal en index.html" by user "jpaandrango" with commit hash "ea726c4". Below the commit, a review section shows a comment by "jpaandrango" stating "codigo revisado" with a "View reviewed changes" link. At the bottom, a message indicates "jpaandrango merged commit bf9243e into `main` now" with a "Revert" button. A final status bar at the bottom reads "Pull request successfully merged and closed". On the right side, a sidebar lists various filters like "Reviewers", "Labels", "Projects", "Milestones", and "Development", all showing "None yet".

Creación de rama, commit y publicación de cambios — Estudiante A (intercambio de roles)

Para demostrar el intercambio de roles, en esta ronda el Estudiante A asumió el rol de desarrollador: creó la rama `rama_estudiante-A`, agregó un párrafo descriptivo en `index.html`, realizó el commit correspondiente (“Agrega párrafo descriptivo en `index.html`”) y publicó la rama en el repositorio remoto.

```
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-A> git checkout -b rama_estudiante-A
Switched to a new branch 'rama_estudiante-A'
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-A> echo "<p>Esta seccion fue agregada por el Estudiante A.</p>" >> index.html
>>
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-A> git add .
>>
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-A> git commit -m "Agrega parrafo descriptivo en index.html"
>>
[rama_estudiante-A 5e6b28d] Agrega parrafo descriptivo en index.html
1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-A> git push origin rama_estudiante-A
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 423 bytes | 423.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'rama_estudiante-A' on GitHub by visiting:
remote:   https://github.com/jpaandrango/tarea_03_devops/pull/new/rama_estudiante-A
remote:
To https://github.com/jpaandrango/tarea_03_devops.git
 * [new branch]      rama_estudiante-A -> rama_estudiante-A
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-A>
```

Pull Request, revisión, aprobación y Merge — Estudiante B (intercambio de roles)

Con los roles intercambiados, el Estudiante B revisó y aprobó el Pull Request #2 “Agrega párrafo descriptivo en `index.html`”, y ejecutó el Merge hacia la rama `main`, demostrando así que ambos integrantes son capaces de cumplir los roles de creador de rama/Pull Request y de revisor/aprobador.



Generación de un conflicto de integración

Para cumplir con la evidencia obligatoria de resolución de conflictos, se modificó la misma línea del archivo index.html desde dos ramas distintas (cambio-titulo-A, ya fusionada, y cambio-titulo-B). Al crear el Pull Request #4 “Modifica título desde rama de B”, GitHub detectó automáticamente que existía un conflicto y bloqueó la fusión hasta que este fuera resuelto manualmente.



Identificación del conflicto en la terminal

Al actualizar la rama local con `git pull origin main` y luego intentar fusionar `main` dentro de la rama `cambio-titulo-B` mediante `git merge main`, Git reportó explícitamente el conflicto: "CONFLICT (content): Merge conflict in index.html", deteniendo la fusión automática hasta su resolución manual.

```

PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> git pull origin main
>>
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 2), reused 2 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 1.10 KiB | 93.00 KiB/s, done.
From https://github.com/jpaandrango/tarea_03_devops
* branch          main       -> FETCH_HEAD
   3fa1655..c329be7 main     -> origin/main
Updating 3fa1655..c329be7
Fast-forward
 index.html | Bin 188 -> 194 bytes
 1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> git checkout cambio-titulo-B
>>
Switched to branch 'cambio-titulo-B'
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> git merge main
>>
warning: Cannot merge binary files: index.html (HEAD vs. main)
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> 

```

Resolución del conflicto de integración

Se editó manualmente el archivo index.html eliminando las marcas de conflicto generadas por Git y unificando el contenido de ambas ramas. Posteriormente se realizó el commit de la resolución ("Resuelve conflicto de integración entre A y B") y se publicó el cambio con git push origin cambio-titulo-B.

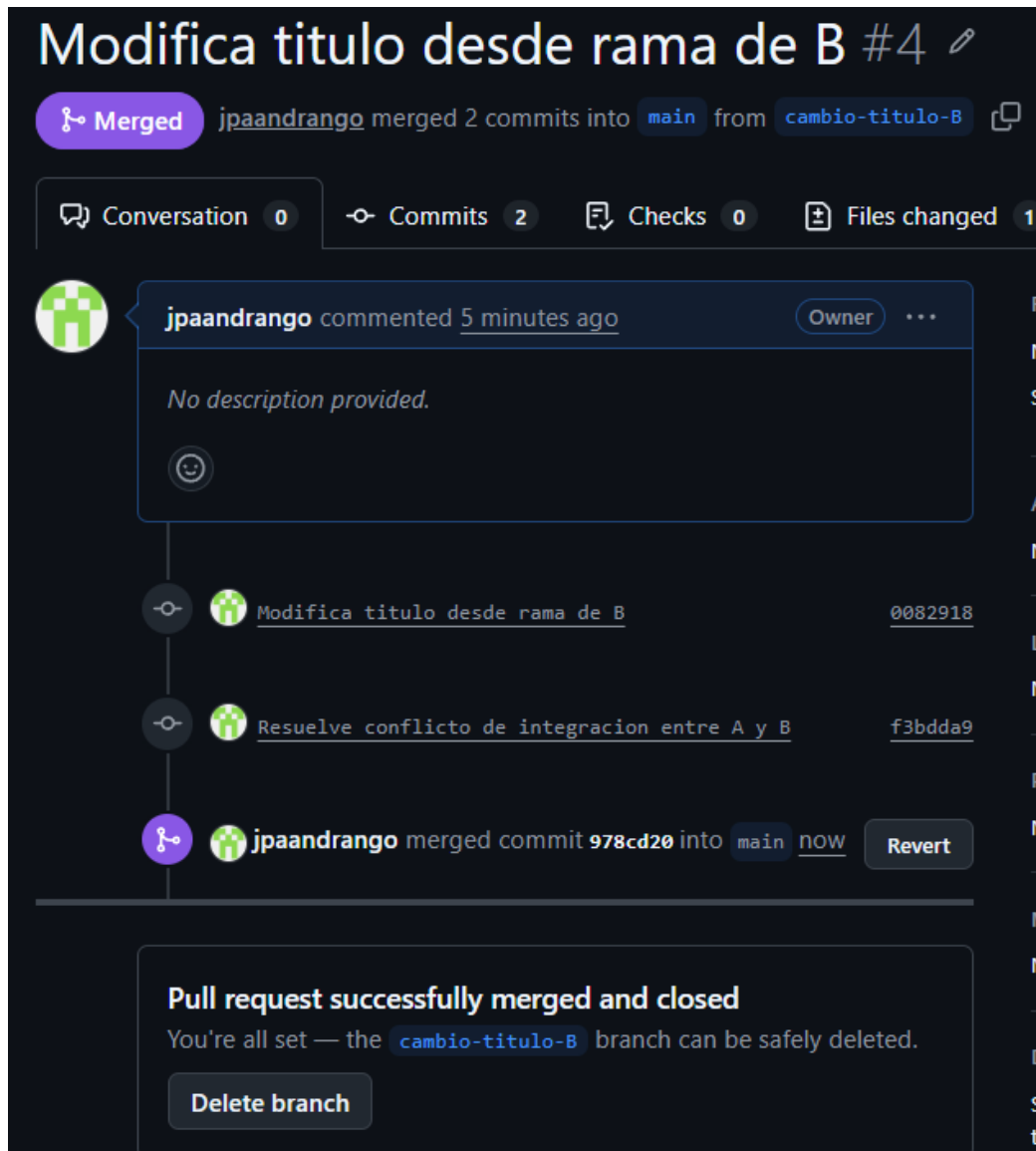
```

>>
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> git commit -m "Resuelve conflicto de integración entre A y B"
[cambio-titulo-B f3bdda9] Resuelve conflicto de integracion entre A y B
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> git push origin cambio-titulo-B
>>
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 370 bytes | 370.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/jpaandrango/tarea_03_devops.git
   0082918..f3bdda9  cambio-titulo-B -> cambio-titulo-B
PS C:\Users\User\Desktop\DEVOPS\tarea_03\estudiante-B> 

```

Merge del Pull Request tras resolver el conflicto

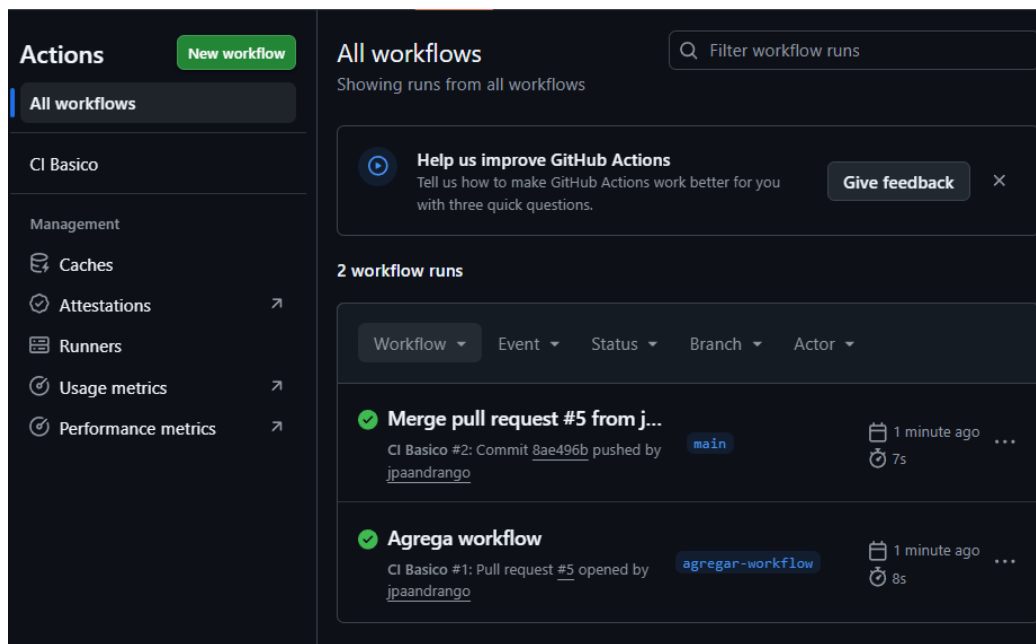
Una vez resuelto el conflicto y publicado el cambio, el Pull Request #4 ya no mostró advertencias de conflicto y pudo fusionarse exitosamente hacia main, incorporando los dos commits de la rama (la modificación original y la resolución del conflicto).



Ejecución del workflow de GitHub Actions

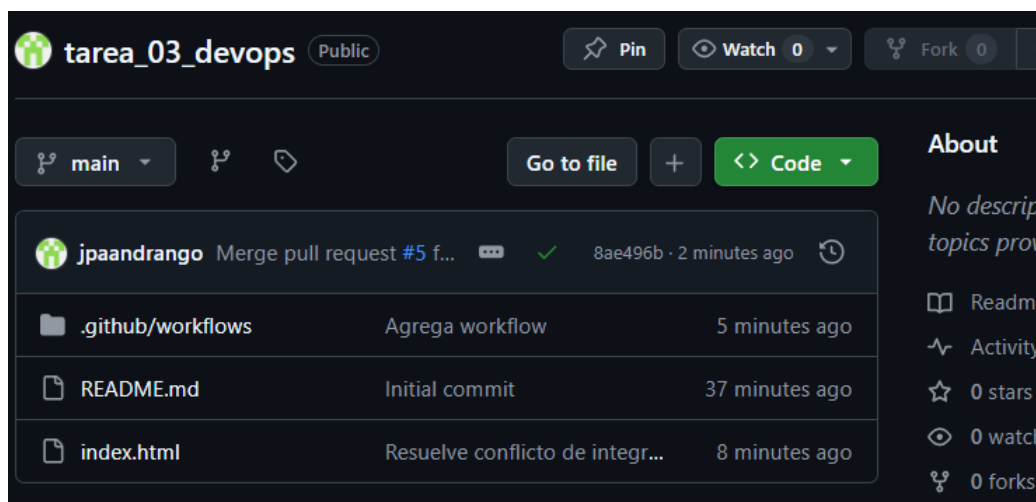
Se configuró un workflow básico de integración continua mediante un archivo YAML ubicado en .github/workflows, programado para ejecutarse automáticamente ante cada push o Pull Request hacia

la rama main. En la pestaña Actions del repositorio se evidencian dos ejecuciones exitosas: una al abrir el Pull Request que agregó el workflow, y otra al fusionarlo hacia main.



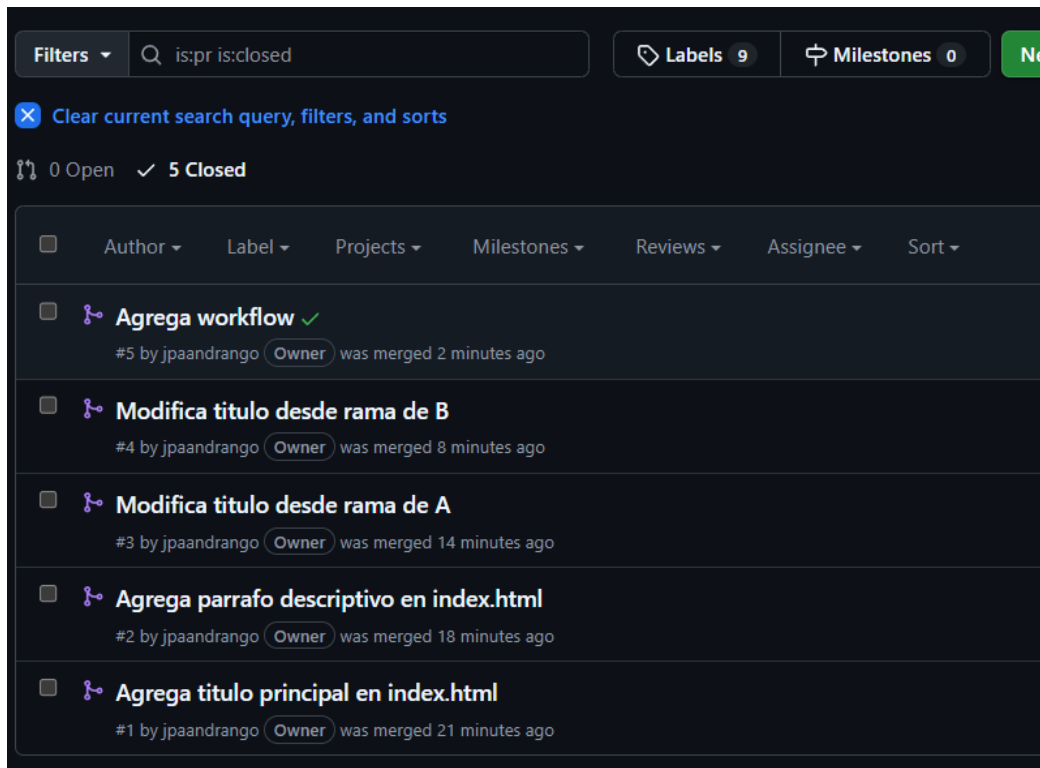
Estado final del repositorio

Tras integrar todos los cambios de ambos integrantes, el repositorio quedó conformado por tres elementos principales: la carpeta `.github/workflows` (con el workflow de Actions), el archivo `README.md` y el archivo `index.html`, este último con el historial de modificaciones realizadas por ambos roles, incluyendo la resolución del conflicto.



Historial de Pull Requests cerrados

Como evidencia de todo el proceso, la pestaña Pull Requests del repositorio muestra los 5 Pull Requests generados durante la práctica, todos fusionados exitosamente (0 abiertos, 5 cerrados): la incorporación del título principal, el párrafo descriptivo, las dos modificaciones que generaron el conflicto (rama de A y rama de B) y la incorporación del workflow de GitHub Actions.



The screenshot shows the GitHub interface for viewing closed pull requests. At the top, there's a search bar with the query 'is:pr is:closed'. Below the search bar, there's a button to 'Clear current search query, filters, and sorts'. The status bar indicates '0 Open' and '5 Closed'. The main list of pull requests is as follows:

☐	Author	Label	Projects	Milestones	Reviews	Assignee	Sort
☐	 Agrega workflow ✓						
	#5 by jpaandrango (Owner) was merged 2 minutes ago						
☐	 Modifica titulo desde rama de B						
	#4 by jpaandrango (Owner) was merged 8 minutes ago						
☐	 Modifica titulo desde rama de A						
	#3 by jpaandrango (Owner) was merged 14 minutes ago						
☐	 Agrega parrafo descriptivo en index.html						
	#2 by jpaandrango (Owner) was merged 18 minutes ago						
☐	 Agrega titulo principal en index.html						
	#1 by jpaandrango (Owner) was merged 21 minutes ago						

Información Adicional

Nombre: Jonathan Andrango

Carrera: Desarrollo de Software

Paralelo: 5 "C" Nocturno

URL del repositorio: https://github.com/jpaandrango/tarea_03_devops.git

Conclusion:

Durante esta práctica aprendí a utilizar Git y GitHub como herramientas para el control de versiones y la colaboración en proyectos de software. Comprendí la importancia de trabajar con ramas para desarrollar cambios sin afectar la versión principal del proyecto. También practiqué la realización de commits con mensajes descriptivos y la publicación de cambios en un repositorio remoto. Además, aprendí a crear Pull Requests y a gestionar el proceso de integración de código. Esta actividad me permitió entender mejor el flujo de trabajo utilizado en entornos profesionales de desarrollo.

Bibliografía:

Git. (s.f.). About — Git. Recuperado de <https://git-scm.com/about>

Chacon, S. y Straub, B. (s.f.). Pro Git Book (2da ed.). Recuperado de <https://git-scm.com/book/en/v2>

GitHub Docs. (s.f.). About GitHub and Git. Recuperado de <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>

GitHub Docs. (s.f.). About pull requests. Recuperado de <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>

GitHub Docs. (s.f.). Fork a repo. Recuperado de <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo>

GitHub Docs. (s.f.). Resolving a merge conflict using the command line. Recuperado de <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-using-the-command-line>

GitHub Docs. (s.f.). Continuous integration and continuous deployment. Recuperado de <https://docs.github.com/en/actions/concepts/overview/continuous-integration-and-continuous-deployment>

GitHub Docs. (s.f.). Understand GitHub Actions. Recuperado de <https://docs.github.com/en/actions/get-started/understand-github-actions>